Guia: como criar e lançar um jogo no Roblox

Tudo o que aprendemos fazendo o Dog Fighter Simulator

1. Ferramentas e organização

- **Roblox Studio**: onde o jogo abre, é testado e publicado.
- **Rojo**: o código fica em arquivos `.lua` numa pasta normal e vira um arquivo `.rbxl` / `.rbxlx`.

Assim dá para editar tudo fora do Studio, guardar versões e gerar o jogo inteiro de uma vez.

- **Separação do código** (padrão que funcionou bem):
  - `src/server` → regras do jogo, dinheiro, save, lutas. **Tudo que vale Robux ou Cash fica no servidor.**
  - `src/client` → telas, sons, câmera, efeitos, tutorial.
  - `src/shared` → configurações usadas pelos dois (preços, raridades, IDs de música e gamepass).
- **Um arquivo de configuração** (`Config.lua`) com preços, tempos e XP. Balancear o jogo vira trocar números,

não reescrever código.

- **Mapa feito por código** (`MapBuilder`): fácil de mudar e sempre igual. Para editar à mão:

Play → copiar `Workspace.Map` → Stop → colar.

- **Antes de publicar, sempre rodar a checagem** (`CHECK.sh`: erros de código + propriedades + build).

2. Lições técnicas (erros que já aconteceram)

Problema	Causa	Solução
Cachorros mortos não sumiam	`BindableEvent` **copia** as tabelas, então o servidor perdia a referência	Usar um `Signal` em Lua puro entre serviços do servidor, e todo corpo se apaga sozinho depois de X segundos
Resposta do servidor chegava incompleta	A nossa função `Net.Invoke` só repassa 2 valores	Devolver **uma tabela** com tudo dentro
Tutorial travou a tela do ovo	Tutorial estava por cima (DisplayOrder maior)	Tutorial abaixo das telas importantes e escondido enquanto elas estão abertas
Banner de level tampava a tela no celular	Interface feita pensando no PC	Escala menor no celular (`IsTouch`), banners menores e mais curtos
Abrir 170 ovos = 170 animações	Uma animação por ovo	Vários ovos seguidos → grade de 10 por vez + CONTINUE / SKIP ALL
Ovos acumulavam com o jogador offline	O timer contava o tempo fora do jogo	Ao entrar, dá **no máximo 1** ovo pelo tempo offline
Erro de sintaxe misterioso	Vírgula sobrando depois de um valor	Rodar a checagem antes de publicar

Regras gerais:

- **Nunca confiar no cliente**: o cliente só pede ("quero abrir ovo"); o servidor confere e faz.
- **Limitar pedidos** (throttle) para ninguém spammar o servidor.
- **Save**: salvar periodicamente + ao sair; ao carregar, "limpar" os dados (campos novos com valor padrão)

para jogadores antigos não quebrarem quando o jogo é atualizado.

- **Tudo que pode falhar** (DataStore, compras, rankings) vai dentro de `pcall`.
- No Studio, o save só funciona com **Game Settings** → **Security** → **Enable Studio Access to API Services**.

3. Interface e celular

- A maioria dos jogadores do Roblox está no **celular**: testar sempre no emulador de celular do Studio.
- Botões grandes o suficiente, mas telas e avisos **menores** que no PC.
- Escalar a interface pelo tamanho da tela (`ScaleFactor`) e respeitar a área segura (notch).
- **Jogo em inglês** alcança muito mais gente.
- **Menu de Settings** (gráficos, volume da música, volume dos efeitos), salvo no perfil do jogador.

4. Tutorial

- Pouco texto, **mais coisa visual**: seta apontando o botão, caminho dourado no chão,

marcador em cima do objeto.

- Passos curtos (ícone + título + uma frase), bolinhas de progresso e botão **skip**.
- Dar um presente no meio (ovo) e uma recompensa no fim, para o jogador ir até o final.
- Jogadores veteranos não veem o tutorial.

5. Música e efeitos sonoros

- Áudio precisa ser enviado em **create.roblox.com** → **Creations** → **Audio**, e o número (ID) vai no código.
- Contas sem verificação de identidade têm **limite de uploads de áudio por mês**.
- Truque usado: **todos os efeitos num único áudio** (um "pack"), e cada som é um pedaço dele

(`PlaybackRegion`). Um upload só, em vez de dezenas.

- Som em tudo: golpes, mordida, poderes, moedas, cliques, ovos, level up, compras.

Sons de luta saem do lugar no mapa (3D); sons de botão são 2D.

- Música diferente por situação: tema principal, arena, boss + jingles (level up, ovo, vitória).

6. Gamepasses e produtos (como ganhar Robux)

- **Gamepass** = compra uma vez, vale para sempre (2x Cash, VIP, Auto Hatch...).
- **Developer Product** = compra várias vezes (pacotes de Cash, ovo especial...).
- **Ordem certa**: primeiro publicar o jogo, depois criar os passes **dentro daquele jogo**,

copiar os IDs e colocar no código (`Monetization.lua`).

- **Os gamepasses pertencem ao jogo (universo)**. Se publicar como **jogo novo**, os passes antigos

não vão junto: sempre **sobrescrever o mesmo jogo**.

- Cada passe precisa de ícone (512x512), nome, descrição e preço.
- **Itens aleatórios pagos** (ovo pago com cachorro sorteado): alguns países proíbem.

Usar `PolicyService:GetPolicyInfoForPlayerAsync` → `ArePaidRandomItemsRestricted` e

bloquear a compra para esses jogadores. Também é preciso responder isso no questionário do jogo.

- Outras fontes de Robux: Premium Payouts (tempo de jogadores Premium no jogo), roupas/UGC.
- **DevEx** (trocar Robux por dinheiro) exige um mínimo de Robux ganhos (30.000) e outros requisitos.

7. Publicar e atualizar

Pelo Studio: File → Publish to Roblox → (primeira vez) Create new game; depois sempre o **mesmo** jogo.

- Se ficar preso em "Upload in progress": fechar e abrir o Studio, desligar Team Create,

ou usar o publicador automático.

Pelo Open Cloud (publicador `.bat`):

- Criar a chave em **create.roblox.com** → **Open Cloud** → **API Keys**.
- Escolher o jogo **e** marcar as **operações de escrita** (ex.: publicar place, assets, game-passes,

developer-products). Erro **403 "Scope not authorized"** = faltou operação/permissão na chave.

- IP aceito: `0.0.0.0/0`.
- **A chave é uma senha**: não mandar o arquivo para ninguém e trocar a chave se ela vazar.
- A API **não cria jogo novo**; ela atualiza um jogo que já existe.

Depois de publicar: máximo de jogadores certo (o mapa tem 8 casinhas = 8 jogadores),

ícone do jogo, thumbnails, descrição e deixar o jogo **Público**.

8. Público, idade e alcance

- Responder o **questionário de maturidade** com sinceridade (violência de desenho, compras, itens aleatórios).

Isso define a classificação de idade.

- O jogo começa com alcance limitado; a **taxa de publicação reembolsável** e as regras de

qualidade influenciam se o jogo pode aparecer para públicos mais novos e em mais lugares.

Conferir sempre a página de **Audience** no painel do jogo.

- O que faz o Roblox recomendar o jogo é **retenção** (o jogador volta no dia seguinte) e **tempo de jogo**.

9. Design do jogo (o que deixa o jogador voltando)

- **Loop claro**: cachorro → come → sobe de nível → fica forte → ovo → cachorros melhores → arena → Cash → melhorias → bosses.
- **Progressão mais lenta no fim** (o XP necessário cresce mais depois do nível 15) para o jogo não acabar rápido.
- **Timers** (ovo a cada 7 min) fazem o jogador voltar, mas sem acumular offline.
- **Coleção**: imagem de cada cachorro no menu, etiqueta NEW, raridades com cor.
- **Qualidade de vida**: auto venda de repetidos, venda rápida no DOGS, skip de animações longas.
- **Rankings globais** (mais forte de todos + mais vitórias da semana, zerando toda segunda):

dão motivo para competir e voltar.

- Missões, conquistas e bosses para ter sempre um próximo objetivo.

10. Checklist antes de cada atualização

1. Rodar a checagem (0 erros).
2. Testar no Studio: Play, celular (emulador) e 2 jogadores (Test → Clients and Servers).
3. Olhar o Output: nada em vermelho.
4. Publicar no **mesmo** jogo (Studio ou `.bat`).

5. Entrar no jogo de verdade e testar compras, save e o tutorial com uma conta nova.
6. Se algo der errado: tirar print do erro e corrigir.